

Automated E-Learning Data Pipeline

Project Proposal

Overview • Objectives • Scope

Repository: Automated_E-Learning_Data_Pipeline
Document Version 1.0

Table of Contents

- 1. Executive Summary
- 2. Problem Statement
- 3. Project Overview
- 4. Objectives
- 5. Scope
- 6. Out of Scope
- 7. Deliverables
- 8. Success Criteria
- 9. Stakeholders & Beneficiaries
- 10. Assumptions & Constraints

1. Executive Summary

The Automated E-Learning Data Pipeline is a data engineering project that aggregates, cleans, and standardizes online course data from three major e-learning platforms — Coursera, Udemy, and Udacity — into a single, trustworthy analytical data warehouse. The project delivers two interchangeable implementations of the same dimensional model: a local, Docker-based solution orchestrated with Apache Airflow and DuckDB, and a cloud-native solution orchestrated with Microsoft Fabric, dbt, and Power BI.

This pipeline is the foundational data layer for a broader vision: an Intelligent Course Recommendation Platform combining an AI-powered conversational chatbot with a Power BI analytics suite. Before any recommendation logic or analytics can be trusted, the underlying course data must be unified, validated, and structured — which is the explicit purpose of this project.

2. Problem Statement

Students face significant choice paralysis when selecting online courses. With offerings spread across many platforms — each using different naming conventions, pricing models, duration formats, and difficulty labels — learners cannot easily compare options to find courses that match their goals, budget, and skill level.

At the same time, education platforms and downstream systems (such as recommendation engines and analytics dashboards) cannot reliably consume this data in its raw form. Coursera, Udemy, and Udacity each structure course records differently: a 'Beginner' course on one platform may be labeled 'Level 1' on another; durations are expressed as free text in inconsistent units; prices follow one-time or subscription models that are not directly comparable. Without a standardization layer, any analysis or recommendation built on top of this data inherits its inconsistencies — a 'garbage in, garbage out' problem.

This project exists to solve that data quality and consistency problem at its source.

3. Project Overview

The project ingests raw, scraped course catalog data from Coursera, Udemy, and Udacity and transforms it through a layered data pipeline into a Kimball-style star schema: eight dimension tables (platform, domain, level, language, instructor, offering type, date, and offering) and one central fact table (fact_offerings) capturing price, rating, enrollment, duration, and a set of derived analytical metrics.

Two parallel implementations were built to demonstrate the architecture in different environments:

Solution	Orchestration	Storage	Transformation	Reporting
Local	Apache Airflow (Docker)	DuckDB (single file)	dbt-duckdb	Local analysis / dbt docs
Cloud	Microsoft Fabric Data Pipeline	Fabric Warehouse	dbt	Power BI (auto-refreshed)

Both solutions share the same dbt modeling layers (staging, intermediate, snapshots, marts) and the same data quality test suite, ensuring that whichever environment is used, the output data contract is identical.

The pipeline also implements Slowly Changing Dimension (SCD) Type 2 tracking on key course attributes (title, description, skills, URL), preserving a full history of how course content evolves — a requirement for any recommendation system that should not silently lose context behind a past suggestion.

4. Objectives

The project is guided by the following objectives:

4.1 Primary Objectives

1. **Unify cross-platform course data.** Consolidate Coursera, Udemy, and Udacity course catalogs into a single consistent schema, resolving differences in column naming, units, and category labels.
2. **Standardize key attributes.** Normalize difficulty level, language, price model, and duration so that values are directly comparable across platforms (e.g. 'Beginner' means the same thing regardless of source).
3. **Build a dimensional data warehouse.** Structure the cleaned data into a star schema optimized for both BI reporting and machine learning feature extraction.
4. **Track historical change.** Implement SCD Type 2 snapshotting so that changes to course titles, descriptions, skills, and URLs are preserved rather than overwritten.
5. **Enforce data quality automatically.** Validate every pipeline run with automated tests covering uniqueness, null constraints, accepted value ranges, and referential integrity between fact and dimension tables.
6. **Automate end-to-end execution.** Orchestrate ingestion, transformation, and reporting refresh so the pipeline runs without manual intervention, in both a local (Airflow/Docker) and cloud (Microsoft Fabric) environment.
7. **Enable downstream consumption.** Produce a data warehouse that is directly usable by a Power BI analytics layer and by a future recommendation engine / chatbot without further cleaning.

4.2 Strategic Objective

Beyond the immediate technical goals, this pipeline is designed as the reusable, trustworthy data foundation for the broader Intelligent Course Recommendation Platform — supporting both a future AI-powered chatbot (personalized learning roadmaps via LLM-generated queries) and a Power BI analytics product for educators and platform operators.

5. Scope

The scope of this project covers the design, implementation, and orchestration of the data transformation pipeline and its two delivery environments.

5.1 In Scope

- Ingestion of pre-scraped raw course data for Coursera, Udemy, and Udacity (coursera_final_data, udemy_final_data, udacity_final_data).
- A layered dbt project: staging models (1:1 cleaned views per source), intermediate models (combined and business-logic-enriched), and marts (final star schema).
- Standardization logic for level, language, price model, price category, and duration category via dbt macros and CASE-based business rules.
- Derived analytical metrics: popularity_score, engagement_rate_pct, weighted_rating, cost_per_hour, estimated_total_cost, and value_score.
- An eight-dimension, one-fact Kimball star schema (dim_platform, dim_domain, dim_level, dim_language, dim_instructor, dim_offering_type, dim_date, dim_offering, fact_offerings).
- SCD Type 2 historical tracking of course title, description, skills, and URL via a dbt snapshot.
- An automated data quality test suite (uniqueness, not-null, accepted values, referential integrity) executed on every pipeline run.
- Local orchestration via Apache Airflow running in Docker, against a DuckDB warehouse file.
- Cloud orchestration via a Microsoft Fabric Data Pipeline (notebook ingestion → dbt build → Power BI semantic model refresh).
- A Power BI dashboard (.pbix) connected to the resulting star schema, refreshed automatically by the cloud pipeline.
- Project documentation: README files, data modeling diagrams (star schema).

5.2 Out of Scope

- Building the AI-powered chatbot / LangChain recommendation engine described in the broader platform vision.
- Live web scraping or API integration to continuously source new raw course data — the pipeline assumes raw extracts are already available in the source tables.
- User-facing application development (web UI, mobile app, authentication, session management).
- Student-level or engagement-event data (e.g. completions, job placement outcomes) — the current scope is limited to course catalog metadata.
- Production-grade security hardening, multi-tenant access control, or cost-optimization tuning of the Fabric/Azure environment.
- CI/CD automation for the dbt project (e.g. GitHub Actions test/deploy workflows) beyond what is manually configured in Fabric/Airflow.

6. Deliverables

Deliverable	Description
dbt project (local & cloud)	Staging, intermediate, snapshot, and mart models with full test coverage
Local solution	Airflow DAG, Docker Compose setup, DuckDB warehouse, USER_GUIDE.md
Cloud solution	Fabric pipeline definition, dbt project deployed to Fabric, Power BI dashboard
Star schema data warehouse	8 dimension tables + fact_offerings, queryable for BI and ML
SCD2 snapshot	Historical record of course attribute changes over time
Power BI dashboard	PowerBI_Dashboard.pbix, auto-refreshed after each successful cloud run
Documentation	README files for both solutions, data modeling (star schema) reference, this proposal

7. Success Criteria

- All three source platforms are represented in the unified fact table with no data loss beyond documented filtering rules (e.g. invalid course_id).
- 100% of fact table rows resolve to a valid dimension key (no orphaned foreign keys), enforced via relationships tests.
- The full dbt test suite passes on every pipeline run before any downstream system (Power BI) is refreshed.
- Course attribute changes are captured historically via the SCD2 snapshot without overwriting prior states.
- The cloud pipeline runs end-to-end without manual intervention, from raw ingestion through to a refreshed Power BI dataset.
- The resulting warehouse schema is directly consumable by BI tools and is structurally ready to serve as a feature source for a future recommendation engine.

8. Stakeholders & Beneficiaries

Stakeholder	Interest
Students / Learners	Ultimate beneficiaries of consistent, comparable course data that will power future personalized recommendations
Data Engineering Team	Owns and maintains the dbt models, orchestration, and data quality framework
BI / Analytics Consumers	Use the star schema and Power BI dashboard to analyze course performance and trends
Future ML / Recommendation Team	Will consume the warehouse as a feature source for a course recommendation engine
Platform Operators (future)	Education platforms or providers who would adopt the broader recommendation platform

9. Assumptions & Constraints

9.1 Assumptions

- Raw course data extraction (scraping) for Coursera, Udemy, and Udacity has already occurred and is available as structured source tables.
- Source data quality issues (missing fields, inconsistent formats) are expected and are handled defensively within the dbt models rather than corrected at the source.
- The cloud environment (Microsoft Fabric, Power BI) is provisioned and accessible with the necessary workspace permissions.

9.2 Constraints

- `course_id` is only guaranteed unique within a single platform, not globally — all uniqueness logic must use the composite key (`course_id`, `platform`).
- Pricing models differ structurally between platforms (one-time vs. subscription), requiring a normalized `estimated_total_cost` metric rather than direct price comparison.
- The local solution depends on Docker and is intended for development/demonstration; it is not designed for production-scale orchestration.
- The cloud solution's Fabric pipeline is fail-fast and strictly sequential — any failure in ingestion or dbt testing halts the run before Power BI is refreshed, by design.

End of document.